

CloudPhone – Das Handy auf dem Server

Joos Menken

Inhaltsverzeichnis

1	Einleitung.....	2
2	Theoretischer Teil / Grundlagen.....	3
2.1	Hardware.....	3
2.1.1	Smart Phones.....	3
2.2	Software.....	3
2.2.1	Linux.....	3
2.2.1.1	Debian.....	4
2.2.2	C (Programmiersprache).....	4
2.2.3	SSH.....	5
2.2.4	Virtuelle Maschinen.....	5
2.2.4.1	QEMU.....	6
3	Planung und Durchführung.....	7
3.1	Planung des Serversystems.....	7
3.2	Programmierung des Serversystems.....	9
3.2.1	Konzept des Terminals.....	9
3.2.2	Schreiben der Funktionen.....	11
	createuser	11
	startvm	12
3.2.3	Kamera Übertragung (/vm/launch.c).....	16
3.3.1	Grafisches System für das Endgerät (Phone).....	16
3.3.2	Derzeitige Fehler.....	16
4	Fazit und Zusammenfassung.....	17
5	Quellen.....	18
5.1	Information zur Verwendung von KI und Entwicklungsumgebungen.....	18

1 Einleitung

Im Rahmen meines Jugendforschts Projektes „CloudPhone“ (25/26) habe ich mich mit der Entwicklung eines massentauglichen Systems zur zentralen Ausführung von Betriebssystemen auf einem Linux Server beschäftigt. Mein Ziel war es ein System zu schaffen welches ein Betriebssystem auf einem Linux Gerät hostet und per SSH (ähnlich wie Remote Desktop) an ein mobiles End-Gerät spiegelt. Die Besonderheit: Das End-Gerät des Benutzers (z.B.: ein Handy) soll bspw. die Kamera oder die GPS Ortung zurückübertragen.

Da ich ein sehr leistungseffizientes Programm brauche, was nah mit dem Betriebssystem kommunizieren kann, habe ich mich in der Umsetzung für die Programmiersprache C entschieden. In dieser ist u.a. Windows, MacOS, Linux und Unix geschrieben. Als Übertragungs-Protokoll habe ich mich für SSH entschieden, da dieses sehr leichtgewichtig ist und trotzdem stark abgesichert. Da ich zudem ein Serversystem brauchte, was stabil und ohne Abstürze oder Fehler läuft, kam eigentlich nur Linux (Debian) in Frage.

Da ich kein Freund von geschlossenen bzw. zentralisierten Systemen bin, habe ich mich von Anfang an entschieden den gesamten Code zu veröffentlichen und unter eine GPL Lizenz zu stellen, welche die freie Weiterverwendung des Codes erlaubt aber andere dazu zwingt den Code ebenfalls unter eine MIT Lizenz zu freizugeben. Der Code ist einsehbar unter meinem GitHub: <https://github.com/Joosibaeri/cloudphone>

2 Theoretischer Teil / Grundlagen

2.1 Hardware

2.1.1 Smart Phones

Mit Smart Phones (umgangssprachlich auch Handy) bezeichnet man mobile Telefone, die einen Bildschirm mit Touch Funktion eingebaut haben. Diese Geräte unterscheiden sich von früheren Mobiltelefonen dadurch, dass sie nicht nur zum Telefonieren gedacht sind, sondern zahlreiche digitale Funktionen in einem einzigen Gerät vereinen. Smartphones dienen heute unter anderem zur Kommunikation, zur Internetnutzung, zur Medienwiedergabe und als mobile Arbeitsgeräte.^[1]

Einen entscheidenden Wendepunkt in der Entwicklung der Smartphones stellte die Veröffentlichung des ersten iPhones im Jahr 2007 dar. Apple präsentierte ein Gerät, das fast vollständig über einen Touchscreen bedient wurde und auf physische Tasten weitgehend verzichtete. Dadurch wurde eine neue, benutzerfreundliche Art der Bedienung eingeführt, die sich deutlich von den zuvor verbreiteten Tastentelefonen unterschied. Das iPhone kombinierte Telefonie, einen mobilen Internetbrowser und Multimediafunktionen und setzte damit neue Maßstäbe für mobile Endgeräte.^[1]

In den folgenden Jahren entwickelten sich Smartphones sehr schnell weiter. Mit der Einführung von App-Stores wurde es möglich, zusätzliche Anwendungen auf die Geräte zu laden, wodurch Smartphones vielseitig einsetzbar wurden. Gleichzeitig etablierte sich neben dem iPhone mit iOS als Betriebssystem auch Android, ein Betriebssystem, das von verschiedenen Herstellern genutzt wurde und so eine große Auswahl an Smartphones in unterschiedlichen Preisklassen ermöglichte, was maßgeblich zur Verbreitung der Geräte beitrug.^[1]

Technologische Fortschritte wie leistungsfähigere Prozessoren, bessere Kameras, größere und hochauflösende Displays sowie schnellere Mobilfunkstandards führten dazu, dass Smartphones immer mehr Funktionen übernahmen, die zuvor auf mehrere Geräte verteilt waren. Seit den 2010er Jahren haben Smartphones klassische Mobiltelefone weitgehend verdrängt und sind aus dem Alltag nicht mehr wegzudenken. Sie prägen heute die Art, wie Menschen kommunizieren, Informationen abrufen und digitale Dienste nutzen.^[1]

2.2 Software

2.2.1 Linux

Linux bezeichnet ursprünglich den Betriebssystemkern (Kernel), also den zentralen Bestandteil eines Betriebssystems, der die Hardware steuert. Dazu gehören die Verwaltung von Prozessen, Arbeitsspeicher, Dateisystemen, Gerätetreibern und Netzwerkfunktionen. In der Praxis wird Linux fast immer mit zusätzlicher Software kombiniert, sodass ein vollständiges Betriebssystem entsteht. Solche Gesamtsysteme werden als Linux-Distributionen bezeichnet.^[2]

Die Entwicklung von Linux begann 1991, als der finnische Informatikstudent Linus Torvalds einen eigenen Kernel für Personal Computer entwickelte. Ziel war es, ein funktionales und frei verfügbares Betriebssystem für x86-Rechner zu schaffen. Der erste öffentlich veröffentlichte Linux-Kernel erschien im September 1991. Schon früh stellte Torvalds den Quellcode zur Verfügung, wodurch sich ein weltweites Gemeinschaftsprojekt entwickelte.^[2] Ein zentraler Aspekt von Linux ist die Veröffentlichung unter der GNU General Public License (GPL). Diese Lizenz erlaubt es, den Quellcode frei zu verwenden, zu verändern und weiterzugeben – vorausgesetzt, abgeleitete Versionen gewähren dieselben Freiheiten. Dadurch beteiligen sich Einzelpersonen, Universitäten und Unternehmen an der Weiterentwicklung.^[2]

Da der Linux-Kernel allein kein vollständiges Betriebssystem darstellt, wird er in der Regel mit Software aus dem GNU-Projekt sowie weiteren freien Programmen kombiniert. Daraus entstehen Linux-Distributionen, die sich in Aufbau, Zielgruppe und eingesetzter Software unterscheiden. Bekannte Beispiele sind Debian, Ubuntu, Fedora, Arch Linux, openSUSE sowie spezialisierte Distributionen wie Raspberry Pi OS oder Kali Linux.^[2]

Linux läuft auf einer Vielzahl von Hardware-Architekturen – von klassischen Desktop- und Serverrechnern über ARM-Systeme, Embedded-Geräte, Router und Smart-TVs bis hin zu Einplatinencomputern. Zudem bildet der Linux-Kernel die technische Grundlage für Android, das auf Smartphones und Tablets weltweit genutzt wird.^[2]

Im professionellen Bereich ist Linux vor allem als Serverbetriebssystem etabliert, etwa für Webserver, Datenbanken und Cloud-Infrastrukturen. Auch in Supercomputern, IoT-Geräten und Edge-Computing-Umgebungen kommt Linux zum Einsatz. Die Entwicklung des Kernels erfolgt kontinuierlich, mit regelmäßigen Updates für Leistung, Sicherheit und Hardwareunterstützung.^[2]

Die Nutzung von Linux erfolgt wahlweise über eine grafische Benutzeroberfläche oder die Kommandozeile. Viele Distributionen bieten verschiedene Desktop-Umgebungen an, während die textbasierte Steuerung über das Terminal besonders in administrativen und technischen Bereichen eine wichtige Rolle spielt.^[2]

2.2.1.1 Debian

Debian ist ein freies, gemeinschaftlich entwickeltes Betriebssystem, das vom Debian-Projekt gepflegt wird. Es basiert auf dem Linux-Kernel sowie Software des GNU-Projekts und weiteren freien Programmen. Debian

unterstützt zahlreiche Hardware-Architekturen (u. a. x86, ARM, MIPS, PowerPC) und ist in vielen Sprachen verfügbar. Die Software wird als Pakete bereitgestellt, die sich über die Paketverwaltungstools dpkg und APT einfach installieren, aktualisieren und entfernen lassen.^[3]

Das Debian-Projekt wurde 1993 von Ian Murdock ins Leben gerufen, mit dem Ziel, eine offene und gemeinschaftlich gepflegte Linux-Distribution zu schaffen. Der Name „Debian“ ist eine Kombination aus den Vornamen von Ian Murdock und seiner damaligen Freundin Debra Lynn. Seit 1996 tragen stabile Debian-Versionen Codenamen, die an Figuren aus dem Film Toy Story angelehnt sind.^[3]

Die Entwicklung von Debian erfolgt durch eine weltweite Gemeinschaft von Freiwilligen. Entscheidungen werden transparent und nach festgelegten Prozessen getroffen, die in der Debian-Verfassung und weiteren Richtlinien dokumentiert sind. Debian ist nicht nur ein eigenständiges Betriebssystem, sondern dient auch als Grundlage für viele andere Distributionen, darunter Ubuntu.^[3]

2.2.2 C (Programmiersprache)

Die Programmiersprache C ist eine imperative, prozedurale Sprache, die Ende der 1960er- und Anfang der 1970er-Jahre von Dennis Ritchie bei den Bell Laboratories entwickelt wurde. Sie entstand als Weiterentwicklung der Sprache B und wurde vor allem geschaffen, um das Betriebssystem Unix auf verschiedene Rechnerarchitekturen zu portieren. C zeichnet sich durch eine klare, reduzierte Syntax aus und ermöglicht direkten Zugriff auf Speicher und Hardware-Ressourcen. Dadurch eignet sie sich besonders für System- und Anwendungsprogrammierung.^[4]

C ist eine statisch typisierte Sprache, bei der Variablen vor ihrer Verwendung deklariert werden müssen. Der Sprachumfang ist bewusst klein gehalten, was – zusammen mit der Effizienz der erzeugten Programme – maßgeblich zu ihrer weiten Verbreitung beigetragen hat. Zu den zentralen Merkmalen zählen Speicherverwaltung, Zeigerarithmetik und die direkte Manipulation von Speicheradressen.^[4]

Ein weiteres wichtiges Merkmal von C ist die Portabilität des erzeugten Codes: C-Programme lassen sich mit relativ geringem Anpassungsaufwand auf unterschiedlichen Rechnerarchitekturen und Betriebssystemen kompilieren. Aufgrund dieser Eigenschaften wird C vor allem in der Entwicklung von Betriebssystemen, Compilern und Systemsoftware eingesetzt. Viele moderne Programmiersprachen, darunter C++, Python, Java und C#, wurden in C implementiert.^[4]

Um die Kompatibilität und Einheitlichkeit der Implementierungen zu sichern, wurde C im Laufe der Zeit standardisiert. Wichtige Meilensteine sind der ANSI-C-Standard (1980er-Jahre) und spätere ISO-Standards. Diese Normen legen Syntax, Semantik und die Standardbibliothek fest, die grundlegende Funktionen für Ein- und Ausgabe, Zeichenkettenverarbeitung und Speichermanagement bereitstellt.^[4]

2.2.3 SSH

Secure Shell (SSH) ist ein kryptographisches Netzwerkprotokoll, das die sichere Kommunikation über unsichere Netzwerke wie das Internet ermöglicht. Es verschlüsselt die Datenübertragung zwischen einem SSH-Client und einem SSH-Server, sodass sensible Informationen wie Passwörter oder Befehle nicht im Klartext übertragen und von Dritten abgefangen werden können. Entwickelt wurde SSH 1995 von Tatu Ylönen als sicherer Ersatz für ältere, unverschlüsselte Protokolle wie Telnet, rlogin oder rsh, die alle Daten – einschließlich Zugangsdaten – offen übertrugen. Die erste Version (SSH-1) wurde später durch die sicherere und technisch überarbeitete Version SSH-2 abgelöst, die erweiterte Verschlüsselungsverfahren und eine modularere Architektur bietet.^[5]

SSH folgt dem Client-Server-Modell: Der Client baut standardmäßig über TCP-Port 22 eine Verbindung zum Server auf und ermöglicht so sichere Fernanmeldungen, Kommandoausführungen und Dateiübertragungen – stets verschlüsselt und ohne Klartextübertragung. Besonders bewährt hat sich SSH in der Fernwartung von Servern, der Remote-Administration und bei sicheren Datenübertragungen. Zu den wichtigsten Anwendungen zählen Remote-Login, Dateiübertragungen über SFTP oder SCP sowie die Portweiterleitung, bei der beliebige TCP/IP-Verbindungen durch den verschlüsselten SSH-Tunnel geleitet werden. SSH-Implementierungen sind heute für nahezu alle Betriebssysteme verfügbar, ursprünglich vor allem für Unix-ähnliche Systeme, inzwischen aber auch für Microsoft Windows.^[5]

2.2.4 Virtuelle Maschinen

Eine virtuelle Maschine (VM) ist in der Informatik die softwarebasierte Nachbildung eines vollständigen Computersystems, das innerhalb eines physischen Rechners betrieben wird. Sie kapselt virtuelle Hardware, ein Betriebssystem und Anwendungen in einer isolierten Umgebung, sodass die VM für das darin ausgeführte System wie ein eigenständiger, physischer Rechner erscheint – obwohl sie die Ressourcen des Host-Systems nutzt.^[6]

Zwischen der realen Hardware (dem Host) und der virtuellen Maschine liegt eine Abstraktionsschicht, der Hypervisor (oder Virtual Machine Monitor, VMM). Dieser verwaltet die Zuteilung von Ressourcen wie Prozessorzeit, Arbeitsspeicher und Speicherplatz und ermöglicht so den parallelen, unabhängigen Betrieb mehrerer VMs auf einem einzigen physischen Rechner.^[6]

Grundsätzlich werden zwei Typen virtueller Maschinen unterschieden: System-VMs bieten eine vollständige Rechnerarchitektur und erlauben das Ausführen ganzer Betriebssysteme, als würden sie auf echter Hardware laufen. Prozess-VMs hingegen dienen der Ausführung einzelner Programme oder Laufzeitumgebungen unabhängig von der zugrunde liegenden Hardware. Ein bekanntes Beispiel ist die Java Virtual Machine (JVM), die Java-Programme plattformunabhängig ausführt.^[6]

Virtuelle Maschinen basieren auf Virtualisierungstechnologien, die bereits in den 1960er-Jahren entwickelt wurden, um die Nutzung von Computersystemen effizienter zu gestalten. Moderne Virtualisierung ermöglicht es, mehrere Betriebssysteme parallel auf einer Hardwareplattform zu betreiben, ohne dass diese direkt auf die physische Hardware zugreifen. Unterstützt wird dies durch spezielle Funktionen moderner Prozessoren, die die Ausführung mehrerer VMs beschleunigen und effizienter machen.^[6]

2.2.4.1 QEMU

QEMU (Quick Emulator) ist eine freie Virtualisierungs- und Emulationssoftware, die ursprünglich von Fabrice Bellard entwickelt und unter der GNU General Public License (GPL 2) veröffentlicht wurde. QEMU emuliert die gesamte Hardware eines Computersystems, sodass ein Gastsystem innerhalb eines Host-Systems ausgeführt werden kann – ohne direkten Zugriff auf die physische Hardware. Durch dynamische Übersetzung der Prozessorinstruktionen des emulierten Systems in Instruktionen für den Host-Prozessor erreicht QEMU eine hohe Ausführungsgeschwindigkeit.^[7]

Die Software unterstützt eine Vielzahl von Prozessorarchitekturen, darunter x86, PowerPC, ARM, Alpha, MIPS, RISC-V, SPARC und weitere weniger verbreitete Architekturen. Dadurch lassen sich Betriebssysteme und Anwendungen für diese Plattformen auf einem anderen Host-System ausführen.^[7]

QEMU bietet zwei Hauptbetriebsmodi: Im System-Emulationsmodus wird ein vollständiger Computer inklusive CPU, Arbeitsspeicher und Peripheriegeräten simuliert, was das Starten ganzer Betriebssysteme ermöglicht. Im User-Mode können einzelne Programme für eine andere Architektur direkt im Userspace ausgeführt werden – ideal etwa zum Testen plattformfremder Software.^[7]

QEMU läuft auf zahlreichen Host-Betriebssystemen wie GNU/Linux, Windows, BSD, macOS und Haiku. Ein besonderes Feature ist die Möglichkeit, den kompletten Zustand einer virtuellen Maschine zu speichern und auf ein anderes Host-System zu übertragen, wo sie nahtlos weiterläuft (Live-Migration). Unter Unix-ähnlichen Systemen unterstützt QEMU zudem die Emulation von Programmen unterschiedlicher Architekturen im Userspace.^[7]

In Kombination mit Hardware-Virtualisierung (z. B. KVM – Kernel-based Virtual Machine) wird QEMU oft als Geräte-Emulator eingesetzt, während KVM die direkte Ausführung von Gastsystemcode auf der Host-CPU übernimmt. Dies steigert die Leistung deutlich und macht QEMU zu einer leistungsfähigen Lösung für Virtualisierung und plattformübergreifende Emulation.^[7]

3 Planung und Durchführung

3.1 Planung des Serversystems

Um ein System zu entwickeln, das maximale Effizienz, hohe Stabilität und breite Kompatibilität vereint, war die Auswahl der richtigen Technologien von zentraler Bedeutung. Mein Ziel war es, ein System zu schaffen, welches nicht nur ressourcenschonend und ohne unnötige Overheads läuft, sondern auch robust gegen Abstürze ist und sich problemlos skalieren lässt – insbesondere für den parallelen Betrieb mehrerer Geräte auf einem einzigen Server.

Die Wahl der Programmiersprache fiel schnell auf C, da sie wie keine andere Sprache die perfekte Balance zwischen Performance und Kontrolle bietet. C-Code wird direkt in maschinennahe Instruktionen kompiliert, die ohne Laufzeitumgebung oder Interpreter auskommen – ein entscheidender Vorteil gegenüber Sprachen wie Python, Java oder JavaScript. Dadurch entfallen unnötige Overheads, und die Performance bleibt vorhersehbar und zuverlässig. Gleichzeitig ermöglicht C präzisen Zugriff auf Hardware-Ressourcen, was für effiziente Systeme, bei denen jede Millisekunde zählt, unverzichtbar ist. Die Portabilität von C, unterstützt durch standardisierte Bibliotheken wie POSIX, stellt sicher, dass der Code auf fast jeder Hardware läuft, ohne dass aufwendige Anpassungen nötig sind.

Für den Betrieb mehrerer Geräte auf einem Server war Virtualisierung der logische nächste Schritt. Hier entschied ich mich für QEMU, da es nicht nur eine vollständige Hardware-Emulation bietet – inklusive CPU, Speicher und I/O –, sondern auch die Isolation der virtuellen Maschinen garantiert. Sollte eine VM abstürzen oder Sicherheitslücken aufweisen, bleibt der Host und die anderen VMs davon unberührt. Durch die Integration von KVM (Kernel-based Virtual Machine) nutzt QEMU zudem die Hardware-Beschleunigung des Linux-Kernels, was die Performance nahe an native Geschwindigkeit bringt.

Ein zentraler Aspekt des Systems ist die sichere Administration der VMs und des Hosts. Hier kommt SSH (Secure Shell) zum Einsatz, um verschlüsselte Verbindungen zu gewährleisten und den Zugriff auf die Infrastruktur zu kontrollieren. SSH bietet dabei mehrere Vorteile: Alle Kommunikationen zwischen Client und Server sind durch

starke Algorithmen wie AES geschützt. Statt unsicherer Passwörter nutze ich schlüsselbasierte Authentifizierung, die nicht nur sicherer, sondern auch praktischer für automatisierte Prozesse ist. Zudem ermöglicht SSH Port-Forwarding und Tunneling, um Dienste sicher durch verschlüsselte Tunnel zu routen – ideal für die Verwaltung von VMs oder den Zugriff auf interne Systeme. Besonders bei der Entwicklung unter Kali Linux und der Verwaltung der Debian-Hosts ist SSH unverzichtbar: Es erlaubt mir, Änderungen direkt auf den Host oder die VMs zu übertragen, ohne die Stabilität des Systems zu gefährden.

Als Betriebssystem für den Host kam für mich nur Debian Linux infrage. Debian ist bekannt für seine stabilen, konservativen Paketversionen, die ideal für Server sind, die rund um die Uhr laufen müssen. Die regelmäßigen Sicherheitsupdates und die aktive Community minimieren zudem Risiken. Ich selbst habe unter Kali Linux entwickelt und getestet, da dieses auf Debian basiert und somit dieselben Systembibliotheken verwendet.

Mit dieser Kombination aus C, QEMU, Debian und SSH entsteht ein System, das nicht nur effizient und stabil läuft, sondern auch skalierbar und sicher ist. Es vereint die Vorteile direkter Hardware-Kontrolle, robuster Isolation, eines bewährten Betriebssystems und einer sicheren Fernverwaltung.

2.2 Programmierung des Serversystems

3.1.1 Konzept des Terminals

Im ersten Schritt habe ich ein kleines Terminal geschrieben, dieses funktioniert folgendermaßen: Zuerst wird `stdio.h` und `string.h` geladen, `stdio.h` braucht man um Daten ein/aus zu geben (**Standard i/o**), `string.h` wird für die Verarbeitung der Daten verwendet:

```
#include <stdio.h>
#include <string.h>
```

Danach werden die Funktionen definiert. In C muss der Code einer Funktion vor ihrem Aufruf bekannt sein, deshalb werden die Funktionen vor der Befehlsabfrage angelegt:

```
void beispiefunktion() {
    /*Code der Funktion*/
}
```

Im nächsten Schritt wird die Funktion „main“ definiert. Innerhalb dieser Funktion wird ein Zeichenarray mit 128 Zeichen als Puffer für die Eingaben über das Terminal angelegt:

```
int main() {
    char input[128];
```

Daraufhin folgt eine Endlosschleife, die das Terminal dauerhaft aktiv hält. In jeder Iteration wird der Benutzer zur Eingabe aufgefordert, und die Eingabe wird in der Variable `input` gespeichert. Tritt ein Fehler oder das Ende der Eingabe auf, wird die Schleife beendet:

```
while (1) {
```



```
printf("> ");
if (!fgets(input, sizeof(input), stdin)) {
    break;
}
```

Da die Befehle mit Enter eingegeben werden wird das abschließende Zeilenumbruchzeichen entfernt, damit die Eingabe korrekt ausgewertet werden kann. Anschließend wird die Eingabe mit bekannten Befehlen verglichen und bei Übereinstimmung die passende Funktion, z.B. beispielbefehl(), aufgerufen. Ist der Befehl unbekannt, wird eine Fehlermeldung ausgegeben.

```
input[strcspn(input, "\n")] = 0;
} else if (strcmp(input, "Beispielbefehl") == 0) {
    beispielbefehl ();
} else {
    printf("Unbekannter Befehl\n");
}
}
```

In den letzten Zeilen wird nur noch angegeben das das Skript nicht wiederholt werden soll und die Klammer geschlossen:

```
return 0;
}
```

3.1.2 Schreiben der Funktionen

Um die Bedienung zu erleichtern habe ich eine Reihe an Funktionen geschrieben, zu diesen gehören:

- checkuser (prüft ob ein Benutzername bereits belegt ist)
- cloneuser (klont die Betriebssystem-Daten eines Users)
- createuser (erstellt einen Benutzer)
- exit (schließt das Programm nach erneuter Abfrage)
- help (zeigt alle Befehle anzeigen)
- listuser (zeigt alle Benutzer)
- rebuildbase (lädt das verwendete Betriebssystem neu herunter)
- removeuser (löscht einen Benutzer)
- resetuser (setzt die Daten eines Benutzers zurück)
- startvm (startet die Virtuelle Maschine eines Benutzers)
- stopvm (fährt die Virtuelle Maschine eines Benutzers herunter)
- serverip (zeigt die IP-Adresse des Servers an, was wichtig zum Verbinden ist)
- userinfo (zeigt alle wichtigen Informationen eines Benutzers z.B.: Speicherort der Daten oder SSH Port)

Wie man sieht sind manche der Funktionen grau markiert, was daran liegt, dass ich mich in der Erklärung des Codes auf die zwei schwarzen Funktionen begrenzen möchte, da die meisten der grauen Punkte nur aus Standard C Code bestehen. Manche Teile des Codes sind aus Platz und Zeitgründen ausgelassen und mit „[...]“ gekennzeichnet. Zudem wird an manchen Stellen Shell Code verwendet, welcher hier ebenfalls nicht mit zitiert ist.

createuser

Zuerst wird die Funktion definiert:

```
void createUser(void)
```

```
[...]
```

Daraufhin wird ein 128-Byte-Puffer im Arbeitsspeicher für den Benutzernamen reserviert. Das Programm zeigt eine Eingabeaufforderung an und wartet, bis der Benutzer einen Namen eingibt:

```
char name[128];  
printf("Enter new account name: ");  
if (!fgets(name, sizeof(name), stdin)) {  
return;
```

```
}
```

In den nächsten Schritten prüft das Programm den Nutzernamen auf ungültige Zeichen und entfernt das beim Eingeben entstandene `\n` (C speichert ein Enter/Zeilenumbruch mit `\n`):

```
if (snprintf(accountPath, sizeof(accountPath), "%s/%s", ACCOUNTS_DIR, name) >= (int)sizeof(accountPath))
{ printf("Name too long\n"); return; }
DIR *d = opendir(accountPath); if (d) { closedir(d); printf("Error: Account '%s' already exists!\n", name); return; }
}
```

Als letztes wird nur noch das Verzeichnis für den Benutzer erstellt und der Starthelfer (wird später wichtig) kopiert:

```
if (mkdir(accountPath, 0755) != 0 && errno != EEXIST) { perror("mkdir user"); return; }
char diskPath[PATH_MAX]; if (snprintf(diskPath, sizeof(diskPath), "%s/disk.qcow2", accountPath) >= (int)sizeof(diskPath)) return;
if (access(VM_BASE_QCOW2, F_OK) == 0) {
if (copyFile(VM_BASE_QCOW2, diskPath) != 0) fprintf(stderr, "Warning: failed to copy base to %s\n", diskPath);
}
}
```

startvm

Zuerst wird die Funktion wieder definiert:

```
void startVM(void) {
```

Danach wird überprüft ob QEMU (`/usr/bin/qemu-system-x86_64`) bereits installiert ist, wenn nicht wird abgebrochen:

```
if (access("/usr/bin/qemu-system-x86_64", X_OK) != 0) {
fprintf(stderr, "QEMU is not installed or not in PATH\n");
return;
}
```

[...]

Im 3. Schritt fordert die Funktion den Benutzer auf, einen Account auszuwählen (über die Funktion `selectAccount`). Wenn Enter gedrückt wird, wird der Speicherordner des ausgewählten Benutzers gesucht (`./vm/userdata/accounts/username`):

```
char  accountName[128]; if (!selectAccount(accountName)) return; char  diskPath[PATH_MAX];
snprintf(diskPath, sizeof(diskPath), "%s/%s/disk.qcow2", ACCOUNTS_DIR, accountName);
```

Nachdem nun der Speicherort der des Accounts bekannt ist, wird geprüft, ob für den gewählten Account bereits eine VM läuft. Dazu wird die PID-Datei (PID = Process Identifications Number) gelesen und überprüft, ob der entsprechende Prozess noch aktiv ist. Falls ja, wird eine Fehlermeldung ausgegeben und die Funktion beendet. Ist die PID-Datei nur veraltet, wird sie gelöscht:

```
char  pidpath[PATH_MAX];  snprintf(pidpath,  sizeof(pidpath),  "%s/%s/vm.pid",  ACCOUNTS_DIR,
accountName);
pid_t existing = pidfile_read(pidpath);
if (existing) {
    if (process_is_running(existing)) {
        printf("Error: VM for '%s' already running (pid=%d).\n", accountName, (int)existing);
        return;
    } else {
        /* remove stale pidfile */
        unlink(pidpath);
    }
}
```

[...]

Nach ein paar weiteren Kontrollen wird der Launch-Helfer gestartet, dieser ist das Hilfsprogramm, das später für die Camera-Bridge verwendet wird:

```
if (deployLaunchBinary(accountDir) != 0) {
    fprintf(stderr, "Warning: launch helper not deployed to %s\n", accountDir);
}
```

Im nächsten Schritt wird der SSH Port zugewiesen (SSH läuft zwar eigentlich auf Port 22, hier wird aber jeder VM ihr eigener Port zugewiesen) Zunächst wird versucht, einen zuvor gespeicherten Port aus einer Datei zu lesen. Falls ein solcher Port existiert, wird ab diesem Port nach einem freien Port gesucht, andernfalls beginnt die Suche bei Port 2200. Der gefundene freie Port wird dann in die Port-Datei geschrieben, damit er bei zukünftigen Starts wiederverwendet werden kann:

```
int sshPort = -1;
char sshportpath[PATH_MAX];
```

```

        if (snprintf(sshportpath, sizeof(sshportpath), "%s/%s", accountDir, SSH_PORT_FILE) >=
(int)sizeof(sshportpath)) { printf("Internal path too long\n"); return; }

    int storedPort = -1;
    (void)read_int_file(sshportpath, &storedPort);

    if (storedPort > 0) {
        sshPort = findFreePortFrom(storedPort);
    } else {
        sshPort = findFreePortFrom(2200);
    }

    if (sshPort <= 0) { printf("No free ports available for SSH\n"); return; }
    if (write_int_file(sshportpath, sshPort) != 0) {
        fprintf(stderr, "Warning: could not persist ssh port to %s\n", sshportpath);
    }

```

Daraufhin wird der zuvor schon überprüfte Launch Helper gestartet. Diese erhält als bevorzugten Start-Port den SSH-Port plus eins. Falls der Start fehlschlägt, wird eine Warnung ausgegeben, die Funktion aber nicht abgebrochen:

```

    int cameraPort = -1;
    pid_t camPid = startCameraBridge(accountDir, sshPort + 1, &cameraPort);
    if (camPid <= 0) {
        fprintf(stderr, "Warning: camera bridge not started for '%s'\n", accountName);
    }

```

[...]

Nachdem ein Kind-Process gestartet wurde, wird eine Log-Datei geöffnet und stdout sowie stderr darauf umgeleitet:

```

    int fd = open(userLog, O_CREAT | O_WRONLY | O_APPEND, 0644);
    if (fd >= 0) { dup2(fd, STDOUT_FILENO); dup2(fd, STDERR_FILENO); close(fd); }

```

Dann wird festgelegt, wie QEMU gestartet wird: Der SSH-Port wird für alle Netzwerkverbindungen weitergeleitet, als virtuelle Festplatte dient eine QCOW2-Datei, und ein Host-Ordner („Account“) wird als gemeinsames Dateisystem unter dem Namen „hostshare“ für den Gast freigegeben:

```

char portarg[64]; snprintf(portarg, sizeof(portarg), "user,hostfwd=tcp:::%d-:22", sshPort);
char drivearg[PATH_MAX + 64]; snprintf(drivearg, sizeof(drivearg), "file=%s,format=qcow2,if=virtio",
diskPath);
char virtfs[PATH_MAX + 96]; snprintf(virtfs,
sizeof(virtfs), "local,id=hostshare,path=%s,security_model=none,mount_tag=hostshare", accountDir);
char *const argv[] = {
    "qemu-system-x86_64",
    "-m", "512M",
    "-nographic",
    "-net", portarg,
    "-net", "nic",
    "-drive", drivearg,
    "-virtfs", virtfs,
    NULL
};

```

Die Zeile danach führt den eigentlichen exec-Aufruf aus.

Falls dieser erfolgreich ist, wird der Code danach nie erreicht, da der Prozess durch QEMU ersetzt wird:

```
execvp("qemu-system-x86_64", argv);
```

[...]

Der nächste Schritt behandelt den Erfolgsfall. Wenn nichts aus der Pipe gelesen werden kann (`r == 0`), bedeutet dies, dass der exec-Aufruf erfolgreich war und die Pipe durch `FD_CLOEXEC` geschlossen wurde. Die PID wird in die PID-Datei geschrieben und eine Erfolgsmeldung mit allen relevanten Informationen (SSH-Port, optionaler Camera-Port, PID, Disk-Pfad, Log-Pfad) ausgegeben:

```

if (r == 0) {
    FILE *f = fopen(userPid, "w"); if (f) { fprintf(f, "%d\n", pid); fclose(f); }
    int fd = open(userLog, O_CREAT | O_WRONLY | O_APPEND, 0644); if (fd >= 0) close(fd);
    if (cameraPort > 0) {
        printf("VM started: ssh=%d camera=%d pid=%d disk=%s log=%s pidfile=%s (cam pid may be %d)\n",
sshPort, cameraPort, pid, diskPath, userLog, userPid, (int)camPid);
    } else {
        printf("VM started: ssh=%d pid=%d disk=%s log=%s pidfile=%s\n", sshPort, pid, diskPath, userLog,
userPid);
    }
}

```

[...]

Wenn es keine weiteren Fehler in der letzten Überprüfung gab wird abschließend in nochmals die Log-Datei geöffnet und sofort geschlossen, um sicherzustellen, dass sie existiert. Danach wird die Funktion beendet.

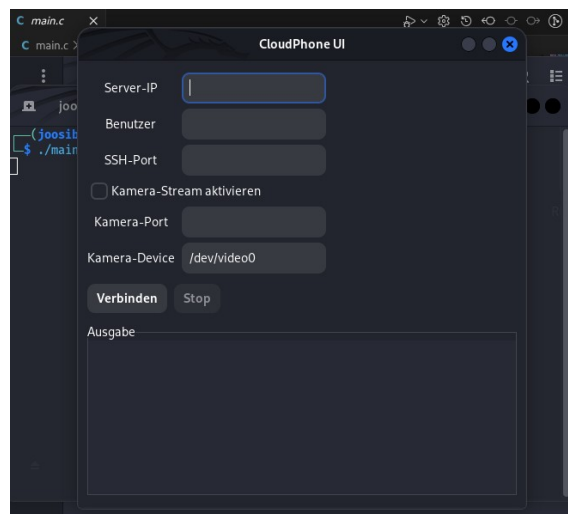
3.1.3 Kamera Übertragung (/vm/launch.c)

launch.c startet einen TCP-Server, der alle eingehenden Datenströme eines konfigurierbaren Ports in eine Datei namens camera.mjpg im Account-Verzeichnis schreibt. Die Datei wird bei jeder neuen Verbindung überschrieben, wobei maximal 50 Megabyte pro Verbindung gespeichert werden. Das Programm arbeitet synchron und behandelt Verbindungen nacheinander, wobei es Logeinträge in cam.log schreibt. Die empfangenen Daten werden indirekt an das virtuelle Betriebssystem weitergegeben: Beim Start der QEMU-VM wird das gesamte Account-Verzeichnis über VirtFS als Share in die virtuelle Maschine eingebunden. Das Gast-Betriebssystem kann diesen Share mounten und hat dann lesenden Zugriff auf die camera.mjpg-Datei. Die VM ist mit Software wie FFmpeg und V4L-Utils ausgestattet, was dafür sorgt, dass eine Kamera-Anwendung innerhalb der VM die MJPEG-Datei kontinuierlich ausliest und verarbeitet. Der Datenfluss verläuft also vom externen Kamera-Stream über das Launch Programm in eine Host-Datei, die dann durch VirtFS für die VM zugänglich gemacht wird.

3.3.1 Grafisches System für das Endgerät (Phone)

Um dem Endbenutzer eine einfach zu bedienendes Programm zu bieten habe ich als letztes Programm noch eine grafische Oberfläche geschrieben, dieses nutzt für die Darstellung GDK:

Diese fragt dann nach der Server IP, dem SSH Benutzer und dem SSH Port, wenn der Checkmarker bei Kamera Stream aktiviert ist sollten auch noch diese Felder ausgefüllt werden. Wenn alles ausgefüllt ist und auf Verbinden gedrückt wird, wird automatisch die Verbindung hergestellt.



3.3.2 Derzeitige Fehler

Leider funktioniert die SSH-Übertragung noch nicht richtig, meiner bisherigen Analyse nach wird zwar die VM einwandfrei gestartet aber noch keine SSH Ausgabe ausgegeben, unabhängig davon ob ich SSH per Termianl oder selbstprogrammierter UI gestartet habe. Meine aktuelle Lösungsidee ist, dass ich SSH statt über den Server, über die VM starte, damit wäre es dann sogar möglich die Limitierung der maximale Anzahl der Geräte pro Server aufzuheben (da über die IP der VM gehostet wird).

4 Fazit und Zusammenfassung

Mit CloudPhone habe ich ein zentralisiertes System entwickelt, was die Kontrolle nur noch ein Stück weiter an Big Tech übergeben kann.

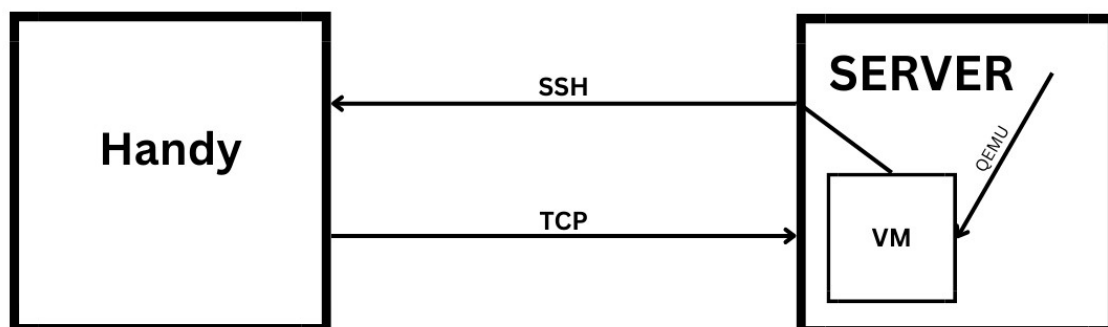
Ich selbst bin kein Freund von zentralisierten Systemen, da dort ein Fehler zu Risiken für eine große Anzahl von Usern führen kann. Schon jetzt könnten Firmen wie Microsoft, Apple und Co. Mit einem Klick unser komplettes Digitales Leben wortwörtlich herunterfahren.

Trotzdem denke ich, dass ein zentrales System auch viele Vorteile haben kann:

Ein modernes Handy hat rund 8-16GB Arbeitsspeicher. Dieser wird aber im Normalfall nicht kontinuierlich gebraucht, erst recht nicht, wenn das Handy ausgeschaltet ist. Genau das war auch meine erste Idee, warum ich mit diesem Projekt angefangen habe. Zudem wird Hardware nicht so schnell unbrauchbar. Man braucht nur ein einfaches Gerät mit Display und sehr wenig Rechenleistung. In dem Moment, wo man das Handy länger nutzen kann, da alle Programme noch flüssig laufen, muss man sich kein Neues kaufen und das Alte entsorgen, was Ressourcen spart.

Ein weiteres Thema was ich hier noch erwähnen möchte ist Künstliche Intelligenz: Apple hat bereits mit „Apple Intelligence“ gezeigt, dass herkömmliche Smartphones zwar stark genug sind um KI auszuführen. Dies erfolgte jedoch in einer so geringen Qualität, dass es zu einer der größten Flops ihrer Geschichte wurde. Wenn nun aber das gesamte System auf einer Cloud betrieben wird, kann dort auch eine hochwertige KI ausgeführt werden und damit deutlich bessere Ergebnisse erzielen.

Für ein besseres Verständnis habe ich final auch nochmal folgende Grafik angefertigt, die zeigt, wie mein System arbeitet:



5 Quellen

^[1] <https://de.wikipedia.org/wiki/Mobiltelefon>

^[2] <https://de.wikipedia.org/wiki/Linux>

^[3] <https://de.wikipedia.org/wiki/Debian>

^[4] [https://de.wikipedia.org/wiki/C_\(Programmiersprache\)](https://de.wikipedia.org/wiki/C_(Programmiersprache))

^[5] https://de.wikipedia.org/wiki/Secure_Shell

^[6] https://de.wikipedia.org/wiki/Virtuelle_Maschine

^[7] <https://de.wikipedia.org/wiki/QEMU>

Die Quellen wurden zuletzt am 04.01.2026 geprüft.

5.1 Information zur Verwendung von KI und Entwicklungsumgebungen

Für die ersten Konzepten von Funktionen wurde in Teilen Künstliche Intelligenz verwendet. Der gesamte Code ist vor Abgabe von mir auf Fehler und möglichen Sicherheitslücken geprüft worden. Die im wesentlichen verwendeten Modelle sind: Mistral 3, GPT-4, GPT-5 mini, GPT-5.1-Codex-Max und Raptor mini, letztere drei wurden in Form von GitHub Copilot verwendet. Zudem wurde mit Inline-Suggestions in VS Code gearbeitet.

Zum vereinfachten schreiben wurde die Entwicklungsumgebung VS Code mit den Erweiterungen: CMake Tools, C/C++ Extension Pack und GitHub Copilot Chat verwendet.